

Runbook: งาน Tester (QA Specialist) — CP353002 Lost and Found

คู่มือทำตามทีละขั้นสำหรับคนที่เพิ่งทำงาน Tester ครั้งแรก ทำตามลำดับ ห้ามข้ามขั้น

เป้าหมายสุดท้าย: `./mvnw test` ผ่านทั้งหมด + กรอก `doc/test-report.md` ครบ + มี ≥ 15 commits ใน branch ตัวเอง

ขั้นที่ 0 — เตรียมเครื่อง (ทำครั้งเดียว)

ต้องมี	ตรวจด้วยคำสั่ง	หมายเหตุ
JDK 17	<code>java -version</code>	ต้องขึ้น 17
Docker	<code>docker ps</code>	จำเป็น เพราะ Integration Test ใช้ Testcontainers (postgres:16-alpine)
Git	<code>git --version</code>	

ตั้งค่า Git ให้ตรงกับบัญชี GitHub ของตัวเอง (ห้ามฝากเพื่อน commit ไม่งั้นได้ 0 คะแนน):

```
git config user.name "ชื่อ-นามสกุลอังกฤษ"
git config user.email "อีเมลที่ผูกกับ GitHub"
```

ขั้นที่ 1 — ดึงโค้ดและเข้า branch ตัวเอง

```
git clone <repo-url>
cd CP353002-Lost-and-Found
git fetch --all
git checkout poranun_6733802763_02 # branch ของ Tester
git merge origin/develop # ดึงงานล่าสุดของทีมมารวม
```

กฎชื่อ branch: ชื่อ_รหัสนักศึกษา_section เท่านั้น ห้ามใช้ชื่ออย่าง `feature/test` (โดนหัก -5)

ขั้นที่ 2 — รันเทสต์ที่มีอยู่ก่อน (Baseline)

```
cd code
./mvnw test
```

อ่านผล:

- BUILD SUCCESS = ผ่าน
- BUILD FAILURE = เลื่อนขึ้นไปดูบรรทัด [ERROR] และไฟล์ใน `target/surefire-reports/`
- ถ้าพังตั้งแต่ก่อนเริ่มเขียนเทสต์ ให้แจ้งทีมก่อน อย่าแก้โค้ดคนอื่นเอง

สำคัญ: โดยค่าเริ่มต้น `mvn test` จะข้าม เทสต์ที่ติด tag `integration` (ตั้งใน `pom.xml`) ถ้าต้องการรันรวม:

```
./mvnw test -DexcludesIntegrationTests=
```

(เครื่องต้องเปิด Docker อยู่ ครั้งแรกจะช้าเพราะดึง image)

ขั้นที่ 3 — ทำความเข้าใจสิ่งที่จะทดสอบ

อ่านโค้ดตามลำดับนี้ (ใช้เวลาประมาณ 1 ชั่วโมง):

1. code/src/main/java/com/example/lostandfound/service/impl/ClaimServiceImpl.java — logic ที่สำคัญที่สุด
2. service/state/* — State Pattern ของสถานะ report (Open / MatchPending / Claimed / Closed / Rejected)
3. service/strategy/* — กฎว่าใครเคลมได้ (Lost / Found)
4. controller/api/* — รายการ endpoint

กฎใน ClaimServiceImpl ที่ต้องมีเทสต์ครอบ:

เมธอด	กรณีที่ต้องทดสอบ	Exception ที่คาดหวัง
submit	report ไม่อยู่ในสถานะเปิดรับเคลม	BadRequestException
submit	เคลมประกาศของตัวเอง	BadRequestException
submit	มีคำขอ PENDING ซ้ำอยู่แล้ว	ConflictException
submit	กรณีปกติ	ได้ ClaimResponse
approve / reject	claim ถูกตัดสินไปแล้ว	BadRequestException
approve / reject	ไม่ใช่เจ้าของประกาศ	ForbiddenException
approve	กรณีปกติ (report เปลี่ยนสถานะ)	สถานะ report ถูกต้อง

ขั้นที่ 4 — เขียน Unit Test (JUnit 5 + Mockito)

ไฟล์: code/src/test/java/com/example/lostandfound/service/ClaimServiceTest.java

โครงสร้างที่ได้เลย (ปรับชื่อ field ให้ตรงกับ constructor จริงของ ClaimServiceImpl):

```
@ExtendWith(MockitoExtension.class)
class ClaimServiceTest {

    @Mock ClaimRepository claimRepository;
    @Mock ReportRepository reportRepository;
    // เพิ่ม @Mock ตาม dependency ที่ ClaimServiceImpl ต้องใช้
    @InjectMocks ClaimServiceImpl claimService;

    @Test
    void submit_ownReport_throwsBadRequest() {
        // Arrange: สร้าง report ที่ reporter คือคนเดียวกับ claimant
        // Act + Assert
        assertThrows(BadRequestException.class,
            () -> claimService.submit(reportId, claimantId, request));
    }
}
```

```
}
```

สูตรทุกเทสต์: **Arrange → Act → Assert** ตั้งชื่อเมธอดแบบ เมธอด_เงื่อนไข_ผลลัพธ์

รันเฉพาะคลาสนี้:

```
./mvnw test -Dtest=ClaimServiceTest
```

Commit ทันทีเมื่อผ่านแต่ละกลุ่ม (ช่วยให้ครบ 15 commits และกระจายตามเวลา):

```
git add .
git commit -m "test: add unit test for ClaimService submit"
git push origin poranun_6733802763_02
```

เทสต์ลำดับถัดไปที่แนะนำ: ReportServiceTest , ตัว State (OpenState ฯลฯ),
ClaimEligibilityStrategyResolver , AuthServiceTest

ขั้นที่ 5 — เขียน Integration Test

ใช้ AbstractIntegrationTest (มี PostgreSQL Testcontainer ให้อยู่แล้ว) ให้ **extends** คลาสนี้:

```
@Tag("integration") // ต้องใส่ ไม่งั้นพฤติกรรมรันไม่ตรงกับที่ pom กำหนด
class ReportRepositoryIT extends AbstractIntegrationTest {

    @Autowired ReportRepository reportRepository;

    @Test
    void contextLoads_andRepositoryWorks() { /* ... */ }
}
```

ข้อควรระวัง:

- application-test.properties ใช้ ddl-auto=create-drop (Hibernate สร้างตารางเอง) ส่วนระบบจริงใช้ validate + schema.sql ถ้าเจอ error เรื่อง schema ให้แจ้งคนดูแล Data Layer
- application.properties ตั้ง active profile เป็น neon (DB บน cloud) ตอนเทสต์ต้องไม่ไปต่อ DB จริง ให้ตรวจว่าเทสต์ใช้ @ActiveProfiles("test") หรือ properties ของ test ทำงานจริง
- อย่าเทสต์โดยใช้ข้อมูลหรือรหัสผ่านจริงของ Neon/Firebase

รันรวม:

```
./mvnw test -DexcludesIntegrationTests=
```

ขั้นที่ 6 — ทดสอบ API ด้วยมือ (Manual API Test)

1. รันระบบ: cd code && ./mvnw spring-boot:run
2. เปิด Swagger UI: <http://localhost:8080/swagger-ui.html>
3. ทดสอบตาม flow นี้ จด ผลจริง ลงตารางทุกครั้ง:

1	สมัครสมาชิก	POST /api/auth/register	200/201
2	ล็อกอิน (เก็บ token)	POST /api/auth/login	200 + token
3	สร้างประกาศ	POST /api/reports	200/201
4	ดูประกาศ	GET /api/reports , /api/reports/{id}	200
5	ยื่นเคลมด้วยบัญชีอื่น	POST /api/reports/{id}/claims	200/201
6	อนุมัติเคลมโดยเจ้าของ	PATCH /api/claims/{id}/approve	200
7	ปิดประกาศ	POST /api/reports/{id}/close	200

กรณีผิดพลาดที่ต้องลองทุกครั้ง:

- ไม่ใส่ token → 401
- ใส่ token แต่ไม่มีสิทธิ์ (เช่น USER เรียก /api/admin/reports/{id}) → 403
- ใช้ id ที่ไม่มี → 404
- ยื่นเคลมซ้ำ → 409
- ส่ง body ว่าง/ผิดรูปแบบ → 400

ขั้นที่ 7 — วัด Coverage

`./mvnw test -DexcludesIntegrationTests=`

JaCoCo ตั้งไว้ให้สร้างรายงานอัตโนมัติตอน test เปิดไฟล์: `code/target/site/jacoco/index.html` ในเบราว์เซอร์แล้ว
กดเปอร์เซ็นต์รวมและของ service

ขั้นที่ 8 — กรอก Test Report

ไฟล์: `doc/test-report.md`

1. เปิด `code/target/surefire-reports/` ดูจำนวน Tests run / Failures / Errors
2. กรอกตาราง "สรุปผลรวม": Total, Passed, Failed, Coverage
3. กรอกตารางรายละเอียดทีละ Test Class: ชื่อคลาส, ชื่อเทสต์, Unit/Integration, ผ่าน/ไม่ผ่าน, หมายเหตุ
4. ใส่ผล Manual API Test (ตารางขั้นที่ 6) เป็นภาคผนวก
5. Commit: docs: fill test report

ขั้นที่ 9 — แจ้งบั๊กและเปิด PR

แจ้งบั๊ก (ส่งใน LINE/Issue ของทีม) ใช้รูปแบบนี้:

[BUG] ชื่อสั้น ๆ
ขั้นตอน: 1) ... 2) ...
คาดหวัง: ...
ได้จริง: ...
หลักฐาน: ภาพหน้าจอ / log

เปิด PR:

1. git push origin poranun_6733802763_02
2. บน GitHub: Pull Request จาก branch ตัวเอง → develop (ห้ามเข้า main ตรง)
3. ขอ Reviewer อย่างน้อย 1 คน (ใบงานแนะนำให้ฝั่ง Backend Data Layer / Git Coordinator ช่วยเช็ค)
4. รอ Review แล้วแก้ตามคอมเมนต์

ขั้นที่ 10 — ก่อนวัน Demo (Checklist)

- ☐ ./mvnw test ผ่านทั้งหมดบน develop ล่าสุด
- ☐ doc/test-report.md ไม่มีช่องว่างเหลือ
- ☐ ซ้อม Demo flow: แจ้งของหาย → ยื่นเคลม → อนุมัติเคลม
- ☐ เตรียมบัญชีทดสอบ 2 บัญชี (เจ้าของประกาศ / ผู้เคลม) ล่วงหน้า
- ☐ commit ของตัวเอง ≥ 15 รายการ กระจายตลอดช่วงโปรเจกต์ (ตรวจด้วย `git log --author="ชื่อ" --oneline | wc -l`)
- ☐ ทุก commit ใช้รูปแบบ test: / docs: / fix: ตามที่อาจารย์กำหนด

ตารางแก้ปัญหาเบื้องต้น

อาการ	สาเหตุที่พบบ่อย	วิธีแก้
Could not find a valid Docker environment	Docker ไม่ได้เปิด	เปิด Docker Desktop แล้วรันใหม่
เทสต์ integration ไม่ถูกรัน	ถูก exclude โดยค่าเริ่มต้น	เติม <code>-DexcludesIntegrationTests=</code>
Schema-validation: missing table	ระบบ/ test ใช้ ddl-auto=validate แต่ไม่มีตาราง	ตรวจว่าเทสต์โหลด application-test.properties และแจ้งคนดูแล schema
Mockito UnnecessaryStubbingException	mock ไว้แต่ไม่ได้ถูกเรียก	ลบ stub ที่ไม่ใช่
NullPointerException ใน @InjectMocks	ลืมน @Mock dependency	เพิ่ม @Mock ให้ครบตาม constructor
push แล้วโดนปฏิเสธ	branch ตามหลัง remote	<code>git pull --rebase origin poranun_6733802763_02</code>

